

Aula Completa

Geolocalização e Mapas Open Source com React Native e Expo

Objetivos da Aula

Ao final desta aula, o aluno será capaz de:

- Entender **como o GPS funciona** em dispositivos móveis e como o React Native acessa esse recurso
- Usar o módulo `expo-location` para obter a posição atual e rastrear o usuário
- Solicitar **permissões de localização** corretamente para Android e iOS
- Exibir um **mapa interativo open source** (OpenStreetMap) usando `react-native-maps`
- Combinar **geolocalização + mapa** para criar um app de rastreamento em tempo real



O que é Geolocalização?

Geolocalização é a capacidade de **determinar a posição geográfica** de um dispositivo no planeta.

Um ponto no mapa é representado por duas coordenadas:

- **Latitude:** eixo norte-sul. Varia de -90° (Polo Sul) a $+90^\circ$ (Polo Norte).
- **Longitude:** eixo leste-oeste. Varia de -180° a $+180^\circ$.

Exemplo: Maringá, PR → Latitude: **-23.42**, Longitude: **-51.93**

Como o Celular Sabe Onde Está?

O Sistema Operacional combina **três fontes** para determinar a localização:

Fonte	Precisão	Velocidade	Consumo de Bateria
GPS (Satélite)	~3–5 metros	Lenta (cold start)	Alto
Wi-Fi	~15–40 metros	Rápida	Baixo
Torres de Celular	~100–1000 metros	Imediata	Baixíssimo

O SO decide automaticamente qual fonte usar — ou combina todas — para dar a **melhor estimativa possível**.



A Arquitetura: Mesmo Princípio da Câmera

Assim como a câmera, o **JavaScript não acessa o GPS diretamente.**

O fluxo é idêntico:

1. **Camada JS (React Native):** Você chama

```
Location.getCurrentPositionAsync()
```

2. **JSI (JavaScript Interface):** A chamada é traduzida em tempo real via memória compartilhada

3. **Camada Nativa (Swift/Kotlin):** O código nativo consulta o `CoreLocation` (iOS) ou `FusedLocationProvider` (Android)

4. **Hardware:** O chip GPS/antenas do celular respondem

5. **Retorno:** As coordenadas chegam de volta como um objeto JavaScript

O Módulo: `expo-location`

O `expo-location` é o **Módulo Nativo** que abstrai toda essa complexidade.

Por baixo dos panos ele contém:

- Interface JS para você consumir
- Código Swift que usa `CLLocationManager` (iOS)
- Código Kotlin que usa `FusedLocationProviderClient` (Android)

Por isso usamos `expo-location` e não `navigator.geolocation` da Web!

A API web funciona apenas em browsers e não tem acesso às otimizações nativas de bateria.

Permissões: Foreground vs Background

A localização tem **dois níveis** de permissão, ao contrário da câmera:

Foreground (Quando o App Está Aberto)

A permissão mais comum. O app só rastreia enquanto o usuário está usando-o ativamente.

Background (Mesmo com App Fechado)

Permissão extra e sensível. Necessária para apps de corrida, delivery, rastreamento de frotas.

A Apple e Google analisam rigorosamente apps que pedem permissão de background. Justifique o uso com clareza.

Configurando Permissões no `app.json`

```
{
  "expo": {
    "plugins": [
      "expo-location",
      {
        "locationAlwaysAndWhenInUsePermission": "Este app usa sua localização para exibir sua posição no mapa.",
        "locationWhenInUsePermission": "Este app precisa da sua localização para funcionar."
      }
    ]
  }
}
```

No Android, o Expo gerencia o `AndroidManifest.xml` automaticamente no build.

Instalação

```
npx expo install expo-location
```

E para o mapa open source:

```
npx expo install react-native-maps
```

O `npx expo install` garante versões compatíveis com o seu SDK atual. Não use `npm install` para módulos nativos!



Parte 1: Obtendo a Localização Atual

Arquivo: App.tsx

```
import { useState, useEffect } from 'react';  
import { View, Text, StyleSheet, ActivityIndicator } from 'react-native';  
import * as Location from 'expo-location';
```

```
type Coordenadas = {  
  latitude: number;  
  longitude: number;  
  precisao: number | null;  
};  
  
export default function App() {  
  const [coords, setCoords] = useState<Coordenadas | null>(null);  
  const [erro, setErro] = useState<string | null>(null);  
  const [carregando, setCarregando] = useState(true);  
}
```

```
useEffect(() => {
  (async () => {
    // 1. Pedir permissão de Foreground
    const { status } = await Location.requestForegroundPermissionsAsync();

    if (status !== 'granted') {
      setErro('Permissão de localização negada pelo usuário.');
```

setCarregando(false);

return;

}

// 2. Obtém a posição atual (usa GPS + Wi-Fi)

const posicao = await Location.getCurrentPositionAsync({

 accuracy: Location.Accuracy.High, // Usa GPS de alta precisão

});

setCoords({

 latitude: posicao.coords.latitude,

 longitude: posicao.coords.longitude,

 precisao: posicao.coords.accuracy,

});

setCarregando(false);

})();

}, []);

```

if (carregando) return (
  <View style={styles.centro}>
    <ActivityIndicator size="large" />
    <Text>Obtendo localização...</Text>
  </View>
);

if (erro) return (
  <View style={styles.centro}>
    <Text style={styles.erro}>✖ {erro}</Text>
  </View>
);

return (
  <View style={styles.centro}>
    <Text style={styles.titulo}>📍 Sua Localização</Text>
    <Text>Latitude: {coords?.latitude.toFixed(6)}</Text>
    <Text>Longitude: {coords?.longitude.toFixed(6)}</Text>
    <Text style={styles.precisao}>
      Precisão: ±{coords?.precisao?.toFixed(0)}m
    </Text>
  </View>
);
}

```

Entendendo o Código

Código	O que acontece por baixo?
<code>requestForegroundPermissionsAsync()</code>	Abre o diálogo nativo do iOS/Android pedindo permissão
<code>getCurrentPositionAsync()</code>	Dispara uma consulta única ao GPS/Wi-Fi/Torres
<code>Location.Accuracy.High</code>	Instrui o nativo a usar o chip GPS (mais lento, mais preciso)
<code>posicao.coords</code>	Objeto retornado do Módulo Nativo com lat, lng, altitude, velocidade, etc.



Parte 2: Mapas Open Source com OpenStreetMap

Por que Open Source?

	Google Maps	OpenStreetMap
Custo	Pago após cota gratuita	Gratuito
Licença	Proprietária	Open Data (ODbL)
Dados	Google controla	Comunidade global
Privacidade	Rastreia usuários	Sem tracking

O `react-native-maps` suporta **ambos**. Para usar o OpenStreetMap, basta trocar o provedor de tiles.



Exibindo o Mapa

```
import MapView, { Marker, UrlTile, PROVIDER_DEFAULT } from 'react-native-maps';
import { StyleSheet, View } from 'react-native';

export default function MapaSimples() {
  return (
    <View style={{ flex: 1 }}>
      <MapView
        style={{ flex: 1 }}
        provider={PROVIDER_DEFAULT} // Não usa Google Maps
        initialRegion={{
          latitude: -23.4205,
          longitude: -51.9333,
          latitudeDelta: 0.05, // Zoom: quanto menor, mais perto
          longitudeDelta: 0.05,
        }}
      >
        { /* Tiles do OpenStreetMap */ }
        <UrlTile
          urlTemplate="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
          maximumZ={19}
          flipY={false}
        />

        { /* Marcador fixo */ }
        <Marker
          coordinate={{ latitude: -23.4205, longitude: -51.9333 }}
          title="Unipar Maringá"
          description="Desenvolvimento de Sistemas"
        />
      </MapView>
    </View>
  );
}
```


Conceitos do Mapa

`initialRegion` e o sistema de Delta

O mapa usa um sistema de **região**, não de zoom numérico:

- `latitudeDelta` : quantos graus de latitude ficam visíveis.
- `longitudeDelta` : quantos graus de longitude ficam visíveis.

```
latitudeDelta = 0.005   → Nível de rua  
latitudeDelta = 0.05    → Nível de bairro  
latitudeDelta = 0.5     → Nível de cidade  
latitudeDelta = 5.0     → Nível de estado
```

`UrlTile`

Carrega os tiles (imagens de 256x256px) diretamente do servidor do OpenStreetMap.

Parte 3: Combinando Geolocalização + Mapa

O objetivo: **mover o mapa e um marcador** conforme o usuário se move.

```
import { useState, useEffect, useRef } from 'react';
import { View, StyleSheet } from 'react-native';
import * as Location from 'expo-location';
import MapView, { Marker, UrlTile, PROVIDER_DEFAULT, Region } from 'react-native-maps';
```

```
export default function MapaComGPS() {  
  const [regiao, setRegiao] = useState<Region>({  
    latitude: -23.4205,  
    longitude: -51.9333,  
    latitudeDelta: 0.01,  
    longitudeDelta: 0.01,  
  });  
  const [posicaoUsuario, setPosicaoUsuario] = useState<{  
    latitude: number;  
    longitude: number;  
  } | null>(null);  
  
  const mapaRef = useRef<MapView>(null);
```

```

useEffect(() => {
  let subscription: Location.LocationSubscription;

  (async () => {
    const { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== 'granted') return;

    // watchPositionAsync: chama o callback TODA VEZ que a posição muda
    subscription = await Location.watchPositionAsync(
      {
        accuracy: Location.Accuracy.Balanced, // Equilíbrio precisão/bateria
        timeInterval: 3000, // Atualiza a cada 3 segundos no mínimo
        distanceInterval: 10, // Ou a cada 10 metros de movimento
      },
      (novaPosicao) => {
        const { latitude, longitude } = novaPosicao.coords;
        setPosicaoUsuario({ latitude, longitude });

        // Anima o mapa para seguir o usuário
        mapaRef.current?.animateToRegion({
          latitude,
          longitude,
          latitudeDelta: 0.01,
          longitudeDelta: 0.01,
        }, 500); // 500ms de animação
      }
    );
  })();

  // Limpeza: para de rastrear quando o componente desmonta
  return () => subscription?.remove();
}, []);

```

```

return (
  <View style={{ flex: 1 }}>
    <MapView
      ref={mapaRef}
      style={{ flex: 1 }}
      provider={PROVIDER_DEFAULT}
      initialRegion={regiao}
    >
      <UrlTile
        urlTemplate="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
        maximumZ={19}
        flipY={false}
      />

      {/* Marcador que acompanha o usuário */}
      {posicaoUsuario && (
        <Marker
          coordinate={posicaoUsuario}
          title="Você está aqui"
          pinColor="#0080ff"
        />
      )}
    </MapView>
  </View>
);
}

```

`watchPositionAsync` vs `getCurrentPositionAsync`

	<code>getCurrentPositionAsync</code>	<code>watchPositionAsync</code>
Tipo	Consulta única	Escuta contínua
Retorno	Promise com uma posição	Subscription com callback
Uso ideal	Mostrar posição inicial	Rastrear movimento
Limpeza	Não precisa	Obrigatório: <code>subscription.remove()</code>
Bateria	Baixo impacto	Impacto contínuo

⚠ **Sempre** chame `subscription.remove()` no cleanup do `useEffect` para evitar **memory leaks** e consumo desnecessário de bateria!

Níveis de Precisão do expo-location

```
Location.Accuracy.Lowest      // Torres celulares – bateria mínima
Location.Accuracy.Low        // Wi-Fi + Torres – pouco preciso
Location.Accuracy.Balanced    // Padrão – bom equilíbrio ★
Location.Accuracy.High        // GPS ativo – mais preciso
Location.Accuracy.Highest     // GPS máximo – drena bateria
Location.Accuracy.BestForNavigation // Navegação turn-by-turn
```

Regra prática:

- App de clima ou check-in? → `Balanced`
- App de corrida ou ciclismo? → `High`
- Navegação GPS? → `BestForNavigation`

Exercício Prático

Crie um app "**Minhas Marcações**" com as seguintes funcionalidades:

1. **Exibir o mapa** centralizado na posição atual do usuário ao abrir o app
2. **Botão "Marcar aqui"**: adiciona um `<Marker>` nas coordenadas atuais e salva em um `useState` como array
3. **Lista de marcações**: exibe abaixo do mapa as coordenadas de cada ponto salvo
4. **Botão "Limpar tudo"**: remove todos os marcadores do mapa e da lista

Desafio extra: Use `expo-sharing` para exportar as coordenadas como um arquivo `.csv` e compartilhá-lo.

Dicas de Arquitetura

Separe as responsabilidades:

```
/components
  MapView.tsx      → Apenas renderiza o MapView + Tiles + Markers
  ListaMarcacoes.tsx → Apenas renderiza a lista de pontos salvos

/hooks
  useLocalizacao.ts → Contém toda lógica de permissão + watchPosition

/screens
  MapaScreen.tsx   → Junta tudo: usa o hook + renderiza os components
```

Isso segue o padrão da aula de **Context API + Hooks**: lógica separada da UI.

Resumo dos Conceitos

-  **GPS no celular** combina satélite, Wi-Fi e torres — o SO escolhe a melhor fonte automaticamente.
-  **expo-location** é um Módulo Nativo que abstrai `CoreLocation` (iOS) e `FusedLocationProvider` (Android) via JSI.
-  **Permissões** são obrigatórias e divididas em Foreground e Background.
-  `getCurrentPositionAsync` → posição única. `watchPositionAsync` → rastreamento contínuo.
-  **react-native-maps** + `UrlTile` entrega mapas gratuitos e open source via OpenStreetMap.
-  Sempre faça **limpeza de subscriptions** no `useEffect` para evitar memory leaks.

Dúvidas?